

Lecture 3: Binary Search

Classic Binary Search — Exact Match

Problem: We have a sorted array $a[0\dots n-1]$ in non-decreasing order. We want to find an index i such that $a[i] == x$.

Example: Find $x = 13$ in array $[3, 5, 8, 13, 18, 19, 21, 27, 32]$

Algorithm:

1. Keep two pointers l and r representing the current search range
2. While $l \leq r$:
 - Compute $mid = (l + r) / 2$
 - If $a[mid] == x$, return mid
 - If $a[mid] < x$, search the right half ($l = mid + 1$)
 - If $a[mid] > x$, search the left half ($r = mid - 1$)

Time Complexity: At each step, we halve the search space $O(\log n)$

Implementation

```
#include <iostream>
#include <vector>
using namespace std;

int binary_search_exact(vector<int>& a, int x) {
    int l = 0, r = (int)a.size() - 1;
    while (l <= r) {
        int mid = l + (r - l) / 2; // Avoids overflow
        if (a[mid] == x) return mid;
        if (a[mid] < x) l = mid + 1;
        else r = mid - 1;
    }
    return -1; // not found
}
```

```

int main() {
    vector<int> a = {3, 5, 8, 13, 18, 19, 21, 27, 32};
    int x = 13;
    int idx = binary_search_exact(a, x);
    cout << (idx != -1 ? "Found at index " + to_string(idx) : "Not found") << "\n";
    return 0;
}

```

Practice Problem Solution

Problem: [Codeforces Binary Search A](#)

Given array and k queries, check if each element exists.

```

#include <iostream>
#include <vector>
using namespace std;

bool find(vector<int> &a, int x) {
    int l = 0;
    int r = a.size() - 1;
    while (l <= r) {
        int m = (l + r) / 2;
        if (a[m] == x)
            return true;
        if (a[m] < x)
            l = m + 1;
        else
            r = m - 1;
    }
    return false;
}

int main() {
    int n, k;
    cin >> n >> k;

    vector<int> a(n);
    for (int i = 0; i < n; i++)
        cin >> a[i];
}

```

```

int x;
for (int i = 0; i < k; i++) {
    cin >> x;
    if (find(a, x))
        cout << "YES\n";
    else
        cout << "NO\n";
}
return 0;
}

```

Finding the Nearest Element

Goal: Find:

- The largest element $\leq x$ (closest from the left)
- The smallest element $\geq x$ (closest from the right)

Algorithm for Largest $\leq x$:

1. Add $-\infty$ at the start and $+\infty$ at the end conceptually
2. Maintain invariant: $a[l] \leq x$ and $a[r] > x$
3. While $l + 1 < r$:
 - $mid = (l + r) / 2$
 - If $a[mid] \leq x$ move $l = mid$
 - Else $r = mid$

Implementation

```

#include <bits/stdc++.h>
using namespace std;

pair<int,int> nearest_elements(vector<int>& a, int x) {
    int n = a.size();
    int l = -1, r = n; // conceptual  $-\infty$  and  $+\infty$ 

    while (l + 1 < r) {
        int mid = (l + r) / 2;

```

```

        if (a[mid] <= x) l = mid;
        else r = mid;
    }

    int leftVal = (l >= 0 ? a[l] : INT_MIN);
    int rightVal = (r < n ? a[r] : INT_MAX);
    return {leftVal, rightVal};
}

int main() {
    vector<int> a = {3, 5, 10, 11, 13, 18, 25, 27, 31};
    int x = 20;
    auto [leftVal, rightVal] = nearest_elements(a, x);
    cout << "Closest left: " << leftVal << ", closest right: " << rightVal << "\n";
    return 0;
}

```

Binary Search on the Answer

Idea: We are looking for the smallest (or largest) x such that some monotonic condition holds. Instead of searching an array, we search in the range of possible answers.

Example 1: Minimum Square to Fit Rectangles

Problem: We have n rectangles of size $a \times b$. Find the smallest square side length that can fit all rectangles (no rotation).

Monotonic function: $f(x) = (\text{floor}(x / a) \times \text{floor}(x / b) \geq n)$

If a square of size x can fit them, a larger square can too **monotonic**.

```

#include <iostream>
using namespace std;

bool canFit(long long side, long long a, long long b, long long n) {
    return (side / a) * (side / b) >= n;
}

long long minSquareSide(long long a, long long b, long long n) {
    long long l = 0, r = max(a, b) * n; // upper bound

```

```

while (l + 1 < r) {
    long long mid = (l + r) / 2;
    if (canFit(mid, a, b, n)) r = mid;
    else l = mid;
}
return r;
}

int main() {
    long long n = 10, a = 2, b = 3;
    cout << "Minimum side: " << minSquareSide(a, b, n) << "\n";
    return 0;
}

```

Example 2: Maximum Rope Length

Problem: We have n ropes, want k equal pieces, maximize length.

Monotonic function: $f(x)$ = can we cut $\geq k$ pieces of length x ?

Here, if we can cut pieces of length x , we can cut smaller ones too **monotonic decreasing**, so we adapt binary search.

```

#include <iostream>
#include <vector>
#include <algorithm>
#include <iomanip>
using namespace std;

bool canCut(const vector<double>& ropes, double len, int k) {
    long long count = 0;
    for (double r : ropes)
        count += (long long)(r / len);
    return count >= k;
}

double maxRopeLength(vector<double>& ropes, int k) {
    double l = 0, r = *max_element(ropes.begin(), ropes.end());

    for (int iter = 0; iter < 100; iter++) { // enough iterations for precision

```

```

        double mid = (l + r) / 2;
        if (canCut(ropes, mid, k)) l = mid;
        else r = mid;
    }
    return l;
}

int main() {
    int n, k;
    cin >> n >> k;

    vector<double> ropes(n);
    for (int i = 0; i < n; i++) {
        cin >> ropes[i];
    }

    cout << fixed << setprecision(2) << "Max length: " << maxRopeLength(ropes, k) << "\n";
    return 0;
}

// Example input:
// 4 11
// 802
// 743
// 457
// 539

```

Key Takeaways

1. **Binary Search works on sorted data or monotonic conditions**
2. **Search the answer space:** There are examples where we don't search an array but a range of possible answers
3. **Floating point precision:** When dealing with floating point answers, iterate fixed times (e.g., 100 iterations) to avoid infinite loops
4. **Always define clear monotonic function $f(x)$** before implementing binary search on the answer
5. **Time Complexity:** $O(\log n)$ for array search, $O(\log(\text{max-min}) \times \text{function_cost})$ for answer search

Binary Search Variants

Type	Use Case	Template
Exact Match	Find specific element	<code>while (l <= r)</code>
Lower Bound	First element $\geq x$	<code>while (l < r)</code>
Upper Bound	First element $> x$	<code>while (l < r)</code>
Answer Search	Optimize on monotonic function	<code>while (l < r)</code> with condition